TASK 10 – ENFORCE ENCAPSULATION THROUGH PACKAGES AND ACCESS MODIFIERS

Programming in Java – Assignment 1 (CO1) | 3 Marks | K4 – Analyse

1. Introduction

Encapsulation combines data and methods into a single unit while restricting direct access to internal data. In the Campus Smart-Parking System, it protects vehicle registration numbers, parking slot details, ticket numbers and parking duration. Java supports encapsulation through private, default (package-private), protected and public access modifiers. Packages organise related classes and provide an additional level of access control.

2. Package Structure

Package	Classes	Responsibility
parking.model	Vehicle, Car, ParkingSlot, ParkingTicket	Stores vehicle, slot and ticket information.
parking.service	ParkingService	Handles parking operations and validation.
parking.app	Main	Executes the application and displays results.

This structure separates data representation, business logic and application execution. It makes the project easier to maintain, test and extend.

3. Access Modifiers and Justification

Modifier	Application	Justification
private	Registration number, slot status, ticket number and duration.	Prevents uncontrolled external modification and protects object state.
default	Package-specific helper methods, where cross-package access is not required.	Restricts access to classes in the same package.
protected	Vehicle constructor and members intended for subclass access.	Allows subclasses to access inherited functionality, subject to Java's protected-access rules.
public	Required constructors, getters and service methods.	Provides the controlled external API used by other packages.

Fields are kept private wherever possible. Public methods expose only the operations required by other packages. Package-private access is appropriate for implementation helpers that should remain internal to a package.

4. Getters, Setters and Data Validation

Getters provide controlled access to private fields, while setters validate values before modifying object state. Registration numbers must not be null or blank; parking duration cannot be negative; floor and slot numbers must be within valid limits; an occupied slot cannot be assigned another vehicle; and an already-free slot cannot be released again. For example, setDurationMinutes() rejects a negative value by throwing IllegalArgumentException. This prevents invalid data from entering the system.

5. Wrapper Classes and Autoboxing

Java wrapper classes represent primitive data types as objects. Integer wraps int, Double wraps double, Long wraps long and Boolean wraps boolean. The implementation uses Integer for parking coordinates. Autoboxing converts a primitive int into an Integer object, while unboxing converts it back.

```
int slotNumber = 5;
Integer storedSlot = slotNumber; // Autoboxing
int actualSlot = storedSlot; // Unboxing
```

A common risk is comparing wrapper objects using ==, which may compare object references instead of numerical values. Use equals() to compare values. A null wrapper object must also be checked before unboxing to avoid

NullPointerException.

```
Integer a = 200;
Integer b = 200;
System.out.println(a.equals(b));
```

6. Alternative Concept and Complexity Analysis

In an alternative design, fields would be public and validation would be performed by the calling code. For example, a public durationMinutes field could be assigned after an if (duration >= 0) check. Any class could modify the field directly, and every calling location would need to repeat the same validation. If one caller forgets the check, an invalid object state may be created.

Comparison	Encapsulated design	Public-field design
Data access	Through controlled methods.	Direct access.
Validation	Centralised in setters.	Repeated in calling code.
Maintainability	Validation logic is easier to update in one place.	Multiple callers may need changes.
Data protection	Stronger control over object state.	Any caller can assign a value.
Validation time per update	O(1) for a simple check.	O(1) for each repeated check.

Both approaches have O(1) time complexity for a simple validation check. The main difference is not the cost of one comparison; it is where the validation is enforced and how many places must maintain it. Encapsulation reduces repeated checks and the risk of inconsistent validation.

7. Conclusion

The Campus Smart-Parking System uses packages and access modifiers to organise classes and protect sensitive data. Private fields, public getters and validated setters maintain data integrity. Wrapper classes and autoboxing support object-based values, while equals() instead of == prevents incorrect wrapper-object comparisons. Therefore, the encapsulated design is safer, easier to maintain and more suitable for future system extensions.